Runnable in LangChain

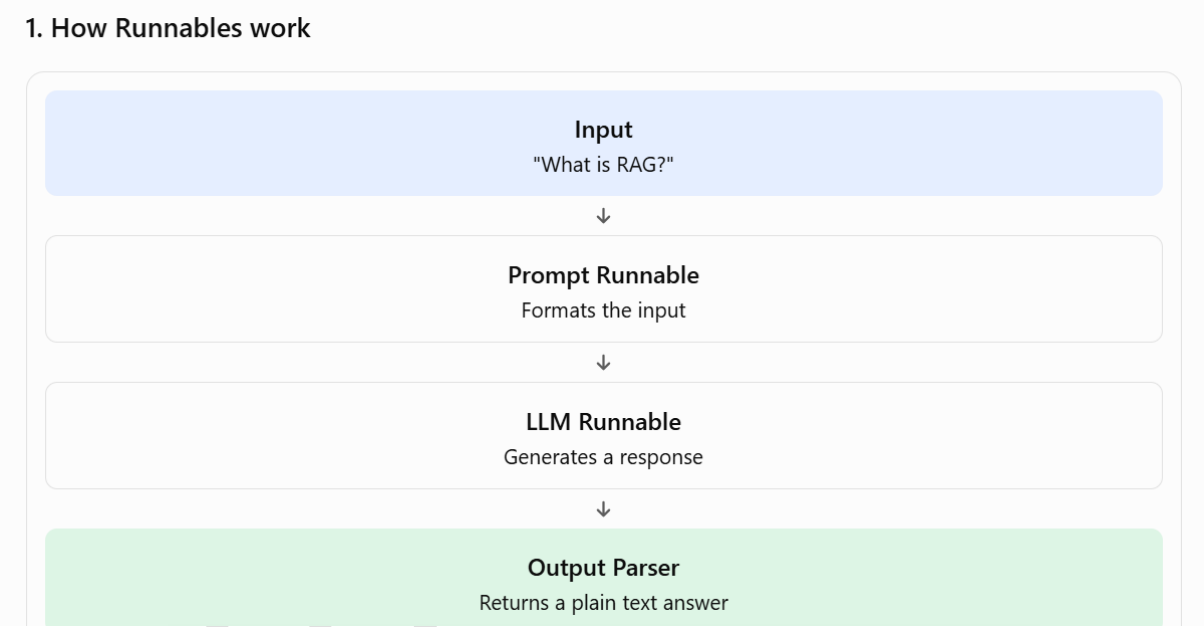
In LangChain, a Runnable is a component that can be executed using a standard interface. Runnables allow you to connect prompts, retrievers, LLMs, output parsers, and custom Python functions into a pipeline.

They are a core part of LangChain Expression Language (LCEL), which uses the | operator to connect components.

For example, in your RAG project, you can connect:

Question → Retriever → Prompt → LLM → Output Parser

Each component performs a task, and the output of one component becomes the input to the next.



Most standard LangChain Runnables support methods such as:

- invoke() — process one input.
- batch() — process multiple inputs.
- stream() — stream output when supported.
- ainvoke() — execute asynchronously.

BY: RAUSHAN KUMAR (https://www.linkedin.com/in/raushan-kumar-553154297)

2. Important Runnable types in LangChain

Runnable	Purpose	Example use
`RunnableSequence`	Executes components sequentially	Prompt → LLM → parser
`RunnableParallel`	Executes multiple branches using the same input	Retrieve context while preserving question
`RunnablePassthrough`	Passes input through unchanged	Preserve the original question
`RunnableLambda`	Converts a Python function into a Runnable	Custom preprocessing or formatting
`RunnableBranch`	Selects a branch based on conditions	Query routing in RAG

3. RunnableSequence — sequential execution

The | operator creates a sequence of operations.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template(
    "Explain {topic} in simple terms."
)

chain = prompt | llm_model | StrOutputParser()

response = chain.invoke({
    "topic": "RAG"
})

print(response)
```

Execution Flow: prompt -> LLM -> String Output

Note: Each stage receives the previous stage's output.

4. RunnableParallel — execute multiple branches

The branches use the same input independently. The results are collected into one dictionary, which can then be passed to a prompt. Below is E2E example.

Define imports:

```
import os
from operator import itemgetter
from langchain_core.documents import Document
from langchain_core.messages import HumanMessage, AIMessage
from langchain_core.prompts import (
  ChatPromptTemplate,
  MessagesPlaceholder
)
from langchain_core.runnables import RunnableParallel
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS
```

Configure the embedding model and LLM

```
embeddings = OpenAIEmbeddings(
  model="text-embedding-3-small"
)
llm_model = ChatOpenAI(
  model="gpt-4o-mini",
  temperature=0
)
```

RAUSHAN KUMAR

Create sample documents

```
documents = [
  Document(
    page_content=(
      "Z-Ordering in Delta Lake co-locates related "
      "column values in data files. It can improve "
      "data skipping and reduce the amount of data "
      "scanned when queries filter on those columns."
    ),
    metadata={"topic": "Z-Ordering"}
  ),
  Document(
    page_content=(
      "OPTIMIZE in Delta Lake compacts small files "
      "into larger files. Z-ORDER BY can be used "
      "with OPTIMIZE to organize data by selected columns."
    ),
    metadata={"topic": "OPTIMIZE"}
  ),
  Document(
    page_content=(
      "A broadcast join in PySpark sends a small "
      "table to worker nodes so that Spark can join "
      "it with a larger table without shuffling "
      "the large table for that join."
    ),
    metadata={"topic": "Broadcast join"}
  )]
```

RAUSHAN KUMAR

Build the FAISS vector store

```
vector_store = FAISS.from_documents(
    documents=documents,
    embedding=embeddings
)
retriever = vector_store.as_retriever(
    search_kwargs={"k": 2}
)
```

Convert retrieved documents into a context string

```
def join_chunks(docs):
    return "\n\n".join(
        doc.page_content for doc in docs
    )
```

```
retrieval_chain = (
    itemgetter("question")
    | retriever
    | join_chunks
)
```

RAUSHAN KUMAR

Define the prompt

```
prompts = ChatPromptTemplate.from_messages([

("system", """You are an Azure Data Engineering assistant. Answer the question using the retrieved context. If the context does not contain the answer, say so.

Retrieved context:

{context}"""),

MessagesPlaceholder(variable_name="chat_history"),

 ("human", "{question}")

])
```

Build RunnableParallel

```
parallel_chain = RunnableParallel(

   question=itemgetter("question"),

   context=retrieval_chain,

   chat_history=itemgetter("chat_history")

)
```

Complete RAG chain

```
full_chain = (parallel_chain | prompts | llm_model | StrOutputParser())
```

Invoke the chain

```
response = full_chain.invoke({

   "question": "What is Z-Ordering and how does it help?",

   "chat_history": [

      HumanMessage(content="I am learning Delta Lake optimization."),

      AIMessage(content="I can explain its optimization techniques.")]})
```

```
print(response)
```

RAUSHAN KUMAR

Important Points:

```
parallel_chain = RunnableParallel(
    question=itemgetter("question"),
    context=retrieval_chain,
    chat_history=itemgetter("chat_history")
)
```

Suppose we invoke it with:

```
inputs = {
    "question": "What is Z-Ordering?",
    "chat_history": []
}
result = parallel_chain.invoke(inputs)
```

Outptu:

The resulting dictionary has this structure:

```
{
    "question": "What is Z-Ordering?",
    "context": "Z-Ordering in Delta Lake co-locates related ...",
    "chat_history": []
}
```

The context value is generated by the retrieval branch. The other two branches extract their values from the original input.

Branch	Input	Output
`question`	Input dictionary	Original question string
`context`	Input dictionary	Retrieved document text
`chat_history`	Input dictionary	Conversation message list

RAUSHAN KUMAR

5. RunnablePassthrough — preserve the input

RunnablePassthrough is a LangChain Runnable that passes the input through unchanged.

Example 1:

```
from langchain_core.runnables import RunnablePassthrough

chain = RunnablePassthrough()

result = chain.invoke("Hello, LangChain!")

print(result)
```

output:

Hello, LangChain!

Example 2: using LCEL

```
from langchain_core.runnables import (
   RunnablePassthrough,
   RunnableParallel
)
chain = RunnableParallel(
   question=RunnablePassthrough(),
   length=lambda x: len(x)
)
result = chain.invoke("What is RAG?")
print(result)
```

Output:

```
{
   'question': 'What is RAG?',
   'length': 14
}
```

RAUSHAN KUMAR

6. RunnableLambda in LangChain

RunnableLambda in LangChain converts a normal Python function into a Runnable so you can use it inside an LCEL chain.

It is useful when you want to add custom Python logic, such as text cleaning, data transformation, or formatting retrieved documents, to a LangChain pipeline.

Example 1:

```
from langchain_core.runnables import RunnableLambda

def uppercase_text(text):
    return text.upper()

chain = RunnableLambda(uppercase_text)

result = chain.invoke("hello langchain")

print(result)
```

Output:

HELLO LANGCHAIN

Example 2: using LCEL (|)

```
from langchain_core.runnables import RunnableLambda

chain = (
    RunnableLambda(lambda x: x.strip())
    | RunnableLambda(lambda x: x.upper())
)

result = chain.invoke("  hello rag  ")

print(result)
```

Output:

HELLO RAG

RAUSHAN KUMAR

7. RunnableBranch in LangChain

RunnableBranch is a LangChain component that executes a particular chain based on a condition. It evaluates conditions in order and runs the first matching branch. If no condition matches, it runs the default branch.

It is useful for query routing in RAG, where different questions need different processing chains.

1. Simple example using LCEL

Suppose you want to route questions based on their type:

- Technical question → Technical response
- Business question → Business response
- Anything else → Default response

```
from langchain_core.runnables import RunnableBranch, RunnableLambda

chain = RunnableBranch(
  (
    lambda x: x["type"] == "technical",
    RunnableLambda(lambda x: "Technical response")
  ),
  (
    lambda x: x["type"] == "business",
    RunnableLambda(lambda x: "Business response")
  ),
  RunnableLambda(lambda x: "Default response")
)

result = chain.invoke({
  "type": "technical",
  "question": "What is a broadcast join?"
})

print(result)
```

Output: Technical response

RAUSHAN KUMAR

Real-world use in RAG

Imagine you are building an Azure Data Engineering assistant.

User question	Route
What is Z-Ordering in Delta Lake?	Databricks documentation retriever
How many records were processed yesterday?	SQL query chain
Explain PySpark broadcast joins	PySpark documentation retriever

You can use RunnableBranch to connect these routes to actual RAG and SQL chains:

```
from langchain_core.runnables import RunnableBranch

router = RunnableBranch(
  (
    lambda x: x["type"] == "databricks",
    databricks_rag_chain
  ),
  (
    lambda x: x["type"] == "sql",
    sql_chain
  ),
  pyspark_rag_chain  # Default branch
)
```

Remember:

RunnableBranch = if / elif / else logic for LangChain pipelines.

RAUSHAN KUMAR